

SimCel

Simulation de Propagation de Feu

Automate cellulaire en plan 2D

Rapport d'activité

Rayân MOUAHID — Sofiane HOUMMASS

CY Tech — Projet Génie Logiciel — 2025–2026

Sommaire

Sommaire.....	2
1. Introduction.....	3
2. Présentation du projet	3
2.1 Périmètre fonctionnel	3
2.2 Contraintes techniques	3
3. Démarche et méthodologie.....	4
3.1 Organisation du travail	4
3.2 Workflow Git	4
3.3 Modélisation UML	4
4. Architecture logicielle.....	5
4.1 Patron MVC et Observer.....	5
4.2 Modèle de propagation	5
4.3 Double buffering.....	5
4.4 Sauvegarde et historique.....	5
5. Déroulement du projet par jalons.....	6
M0 — Setup & Initialisation	6
M1 — Socle du modèle	6
M2 — Moteur de simulation	6
M3 — Paramètres avancés.....	6
M4 — Interface graphique	7
M5 — Finalisation.....	7
6. Problématiques rencontrées.....	7
6.1 Double buffering et effets de cascade.....	7
6.2 Initialisation des cellules aquatiques.....	7
6.3 Gestion de la concurrence GUI / thread de simulation.....	8
6.4 Calcul du facteur vent.....	8
6.5 Refonte de l'interface graphique.....	8
7. Bilan et rétrospective	Erreur ! Signet non défini.
Annexes	9
A. Interface graphique	9
B. Diagramme de cas d'utilisation	11
C. Diagramme d'états-transitions.....	12
D. Diagramme de séquence	13
E. Diagramme de classes	14

1. Introduction

SimCel est un projet réalisé dans le cadre du Projet Génie Logiciel du cursus ingénieur informatique à CY Tech. Son objectif est de développer une application Java complète de simulation d'automate cellulaire, dont la thématique est la propagation d'un incendie dans un environnement naturel.

Le point de départ était une feuille blanche : un sujet, un cahier des charges, et aucune ligne de code. Ce rapport retrace le cheminement complet du projet, des premières décisions d'architecture jusqu'à la livraison d'une application fonctionnelle dotant d'une interface graphique JavaFX et d'un mode console, les problématiques rencontrées et les solutions adoptées.

L'équipe est composée de deux membres :

- Rayan MOUAHID
- Sofiane HOUMMASS

2. Présentation du projet

2.1 Périmètre fonctionnel

L'application SimCel simule la propagation d'un feu de forêt sur une grille rectangulaire 2D. Chaque cellule possède un **état** et un **type de terrain**. À chaque pas de temps (tick), le moteur calcule la propagation du feu selon un modèle stochastique tenant compte du vent et de l'humidité.

Fonctionnalité	Description
Grille paramétrable	Dimensions configurables, de 10×10 à 300×200
6 états de cellule	VIDE, SAIN, EN_FEU, BRÛLÉ, EAU, ROCHER
5 types de terrain	Forêt, Prairie, Broussailles, Zone humide, Zone urbaine
Vent	8 directions (rose des vents), intensité 0–5
Humidité	0 à 100 %, réduction linéaire de la probabilité d'inflammation
Interface GUI	JavaFX : grille Canvas, statistiques, courbe, indicateur de vent
Mode console	Headless via arguments CLI, rendu ANSI
Sauvegarde / Chargement	Format binaire .simcel, snapshot complet de l'état
Historique	Retour arrière jusqu'à 100 ticks en mémoire

2.2 Contraintes techniques

Les contraintes imposées étaient les suivantes : langage **Java**, interface graphique **JavaFX**, gestion de version avec **Git**, production de **diagrammes UML** (cas d'utilisation, classes, séquence, états-transitions), et outil de build **Maven**.

3. Démarche et méthodologie

3.1 Organisation du travail

Dès le début, nous avons structuré le projet en **6 jalons (milestones)**, chacun livrable de façon autonome, progressant du socle du modèle jusqu'à l'interface graphique complète. Ce séquençage garantissait qu'aucun jalon ne nécessitait d'anticiper le suivant.

Milestone	Objectif principal
M0 — Setup	Dépôt, structure Maven, document de spécification, cas d'utilisation
M1 — Socle du modèle	Classes Cell, Grid, Environment, énumérations, diagrammes UML
M2 — Moteur	FireSimulator, SimulationController, ConsoleView, mode headless
M3 — Paramètres avancés	Intégration vent, humidité, configuration interactive en console
M4 — Interface graphique	GUI complète : GridView, StatisticsPanel, ChartView, WindIndicator
M5 — Finalisation	Sauvegarde, chargement, historique, documentation

Chaque issue GitHub constituait une tâche atomique, fermée par un commentaire structuré référençant les fichiers modifiés. Nous avons adopté une discipline constante de **trois phases par issue** : cadrage complet du scope avant toute implémentation, développement, puis documentation et clôture.

3.2 Workflow Git

Le dépôt est organisé autour de trois branches principales :

- **main** : code stable et livrable uniquement
- **dev** (Rayân) : branche de travail individuelle
- **develop** (Sofiane) : branche de travail individuelle

Les fonctionnalités étaient développées en parallèle sur les branches de travail respectives, puis intégrées sur main par pull request. Les messages de commit suivent une convention sémantique (feat, fix, refactor, docs) permettant de retracer l'historique des 96 commits.

3.3 Modélisation UML

Quatre diagrammes UML ont été produits et versés dans le dossier docs/uml/ : diagramme de cas d'utilisation (établi en M0), diagramme de classes (évolutif de M1 à la version finale), diagramme d'états-

transitions (M1), et diagramme de séquence du tick (M2). Ces livrables ont précédé l'implémentation dans chaque cas, servant de contrat avant d'écrire le code.

4. Architecture logicielle

4.1 Patron MVC et Observer

L'application applique strictement le patron **Modèle–Vue–Contrôleur**. La frontière MVC a été un principe directeur tout au long du développement : aucune classe de la couche vue ne contient de logique métier, et le modèle ne connaît pas la vue.

La communication entre le moteur et les vues passe exclusivement par l'interface **SimulationListener**, implémentant le patron Observer. Après chaque tick, FireSimulator appelle notifyListeners(), ce qui déclenche la mise à jour de GridView, StatisticsPanel et ChartView.

Couche	Classes principales
Modèle (model)	Cell, Grid, Environment, FireSimulator, SimulationIO, SimulationSnapshot, SimulationState, WindDirection, CellState, CellType, SimulationListener
Vue (view)	MainWindow, GridView, ConsoleView, StatisticsPanel, ChartView, WindIndicator, LegendPanel, HeaderBar
Contrôleur (controller)	SimulationController

4.2 Modèle de propagation

La probabilité d'inflammation d'une cellule voisine à chaque tick est calculée selon la formule :

```
P = clamp( baseInflammabilité × facteurVent × facteurHumidité, [0 ; 1] )
facteurVent      = 1 + (force / 5) × cos(θ)
facteurHumidité = 1 - humidité / 100
```

θ est l'angle entre le vecteur vent et le vecteur source→cible, calculé par produit scalaire. À force 5, la probabilité est **doublée** dans le sens du vent et **nulle** à contre-vent. À 100 % d'humidité, toute propagation est impossible.

4.3 Double buffering

Le double buffering est la pierre angulaire du mécanisme de tick. En début de chaque pas de temps, un **snapshot** des états courants est capturé sous forme de tableau CellState[[]]. Toutes les décisions de propagation et d'extinction sont prises sur ce snapshot figé, et les modifications sont appliquées à la grille réelle seulement à la fin du calcul. Cela garantit que toutes les cellules partent du même état pour un tick donné, sans effet de cascade lié à l'ordre de parcours.

4.4 Sauvegarde et historique

La classe **SimulationIO** sérialise un objet `SimulationSnapshot` (grille courante, grille initiale, paramètres environnementaux, tick courant) dans un fichier binaire au format `.simcel`. Le chargement vérifie la correspondance des dimensions avant restauration.

En parallèle, `FireSimulator` maintient une **Deque des 100 derniers états** permettant un retour arrière tick à tick, sans accès disque.

5. Déroulement du projet par jalons

M0 — Setup & Initialisation

Ce premier jalon a été consacré entièrement aux fondations : aucune logique métier n'y a été écrite. Nous avons créé et configuré le dépôt GitHub, mis en place la structure Maven avec les packages `model`, `view`, `controller` et `util`, rédigé le document de spécification fonctionnelle détaillant le modèle de simulation, l'architecture cible et les jalons, et produit le diagramme de cas d'utilisation.

La discipline adoptée dès M0, rédiger la spécification avant tout code, a évité les allers-retours d'architecture que l'on rencontre fréquemment dans les projets étudiants.

M1 — Socle du modèle

Ce jalon a implémenté la totalité des classes de la couche Model : les énumérations `CellState` et `CellType` (avec leurs coefficients d'inflammabilité et durées de combustion), la classe `Cell` (gestion des transitions d'état, décrémentement du compteur de combustion), la classe `Grid` (voisinage de Moore, initialisation aléatoire, reset, gestion des coordonnées hors limites), et la classe `Environment` (direction, intensité vent, humidité avec validation des plages).

En parallèle, le **diagramme de classes** (v1) et le **diagramme d'états-transitions** du cycle de vie d'une cellule ont été produits.

M2 — Moteur de simulation

M2 a rendu la simulation fonctionnelle en mode headless. La classe `FireSimulator` a été implémentée avec son algorithme de tick en double buffering. `SimulationController` a été créé pour orchestrer la boucle de simulation dans un thread séparé, avec les commandes `start()`, `pause()`, `step()`, `stop()` et `reset()`. La classe `ConsoleView` a été implémentée pour le rendu ANSI dans le terminal.

La classe `Main` a été adaptée pour analyser les arguments CLI (`--width`, `--height`, `--ticks`, `--delay`, `--wind-direction`, `--wind-strength`, `--humidity`) et lancer la simulation en mode headless.

Le **diagramme de séquence** détaillant le déroulement complet d'un tick a été produit en fin de jalon.

M3 — Paramètres avancés

Ce jalon a intégré le vent et l'humidité dans la méthode `computeInflammationProbability()`. Le facteur vent utilise le cosinus de l'angle entre le vecteur vent et le vecteur de propagation, calculé par produit scalaire. Le facteur humidité est appliqué de façon linéaire. Un mini-interpréteur de commandes console a été ajouté dans `ConsoleView` pour reconfigurer le vent, l'humidité et la vitesse en cours de simulation.

M4 — Interface graphique

M4 représente la part la plus volumineuse du projet. La fenêtre principale `MainWindow` a été structurée autour de trois zones : la grille (gauche), les statistiques et contrôles (droite), et la courbe d'évolution temporelle (bas).

Les composants suivants ont été implémentés :

- **GridView** : rendu sur Canvas JavaFX, zoom de $\times 0,2$ à $\times 8$ centré sur le curseur, panoramique par cliquer-glisser, 9 outils d'édition pour modifier terrain et état de chaque cellule
- **StatisticsPanel** : tick courant, compteurs SAIN/EN_FEU/BRÛLÉ, taux de destruction, mise à jour en temps réel via `SimulationListener`
- **ChartView** : courbe d'évolution sur 3 séries avec fenêtre glissante de 300 points
- **WindIndicator** : boussole animée, sélecteur de direction (ComboBox), sliders force et humidité
- **LegendPanel** : légende des 5 types de terrain et des 6 états
- **HeaderBar** : barre de titre avec indicateur d'état, tick, taille de grille, boutons Sauvegarder/Charger

Une refonte visuelle complète a été réalisée en cours de jalon : remplacement de la palette bleu-violet initiale par un thème sombre neutre avec accent orange ambre (`#f0a030`), refactorisation du CSS JavaFX et des constantes de rendu dans `WindIndicator` et `ChartView`.

M5 — Finalisation

Le dernier jalon a ajouté les fonctionnalités de sauvegarde et chargement (`SimulationIO`, format `.simcel`), l'historique tick-par-tick avec retour arrière (`stepBack()`), et la complétion de la documentation technique (README, Javadoc, mise à jour des diagrammes UML). Le diagramme de classes a été mis à jour pour refléter fidèlement l'implémentation finale.

6. Problématiques rencontrées

6.1 Double buffering et effets de cascade

La première problématique technique majeure a concerné la cohérence de la propagation. Sans mécanisme de protection, une cellule enflammée au tick `N` pouvait immédiatement propager dans ce même tick si ses voisins étaient traités après elle dans la boucle de parcours 2014 créant un effet de cascade artificiel dépendant de l'ordre de parcours.

La solution retenue est le **double buffering** : en début de tick, un tableau `CellState[][]` capture l'état de toutes les cellules. L'algorithme prend toutes ses décisions sur ce snapshot figé, puis applique les modifications à la grille réelle de façon atomique. Toute cellule a ainsi exactement le même statut pour un tick donné, indépendamment de l'ordre de parcours.

6.2 Initialisation des cellules aquatiques

Le constructeur de la classe `Cell` initialise systématiquement l'état à SAIN. Or les cellules aquatiques (EAU) doivent être créées avec cet état spécifique, qui n'est pas un état valide pour un type de terrain classique. Un conflit est apparu lors de l'implémentation de `Grid.initRandom()` : passer EAU comme `CellType` n'avait pas de sens dans le modèle, tandis qu'utiliser le constructeur standard laissait les cellules aquatiques en état SAIN.

La solution adoptée consiste à conserver le constructeur `Cell(CellType)` initialisant à SAIN, et à appeler `setState(EAU)` explicitement dans `Grid.initRandom()` pour les cellules destinées à être aquatiques. Cette approche préserve la sémantique du constructeur tout en permettant l'initialisation particulière des cellules EAU.

6.3 Gestion de la concurrence GUI / thread de simulation

JavaFX impose que toute modification de l'arbre de composants graphiques soit effectuée depuis le **thread JavaFX**. Le thread de simulation, qui appelle `simulator.tick()` à intervalle régulier, est un thread daemon séparé. Les notifications de `SimulationListener` doivent donc être dispatchées vers le thread JavaFX.

La solution est le recours systématique à **`Platform.runLater()`** dans les implémentations de `onTick()` des composants visuels (`GridView`, `StatisticsPanel`, `ChartView`). Cette approche est simple, sûre et suffisante pour des grilles de taille normale.

6.4 Calcul du facteur vent

Le facteur vent devait favoriser la propagation dans la direction du vent et la freiner à contre-vent, de manière proportionnelle à l'intensité. Une première approche par table de correspondance direction-par-direction s'est révélée rigide et incorrecte pour les propagations diagonales.

La solution finale est un **calcul par produit scalaire et cosinus** : le vecteur vent (dx, dy) de `WindDirection` et le vecteur de propagation source→cible sont normalisés, leur produit scalaire donne $\cos(\theta)$, et le facteur s'écrit $\text{windFactor} = 1 + (\text{force}/5) \times \cos(\theta)$. Cette formulation est générique pour toutes les directions et intensités.

6.5 Refonte de l'interface graphique

Lors du développement de M4, l'interface initiale présentait un manque de cohérence visuelle : palette bleu-violet trop colorée, statistiques mal positionnées, indicateur de vent interactif (flèche draggable) générant des conflits d'événements souris avec la grille. Une refonte complète a été entreprise :

- Remplacement de la palette par un thème sombre neutre avec accent orange/ambre (`#f0a030`)
- Passage de l'indicateur de vent en lecture seule (boussole + `ComboBox`)
- Création de la classe `HeaderBar` pour centraliser les informations d'état globales
- Ajout des champs de saisie manuelle des dimensions de grille

Cette itération a illustré un principe général : une conception GUI mérite d'être réévaluée après une première version fonctionnelle, car les besoins d'ergonomie ne se révèlent souvent qu'à l'usage.

Annexes

A. Interface graphique

État initial de la grille (tick 0, mode édition disponible) :

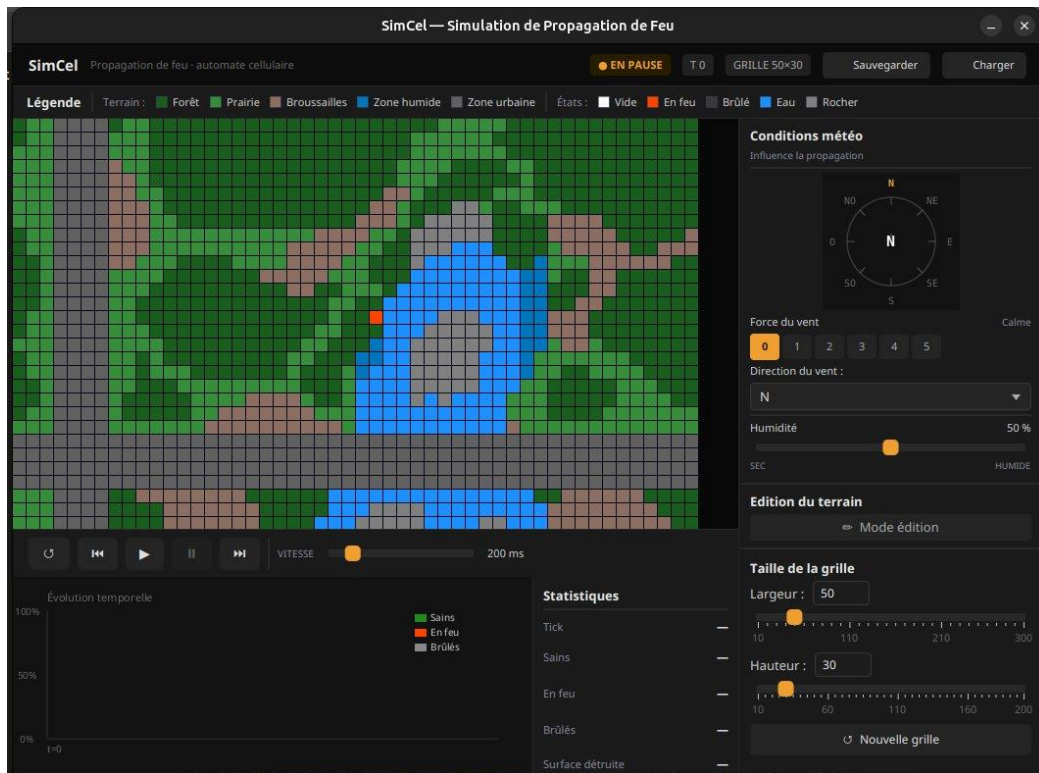


Fig. 1 — SimCel au démarrage, grille 50×30, feu non encore propagé (T 0)

)

Simulation en cours après 168 ticks :

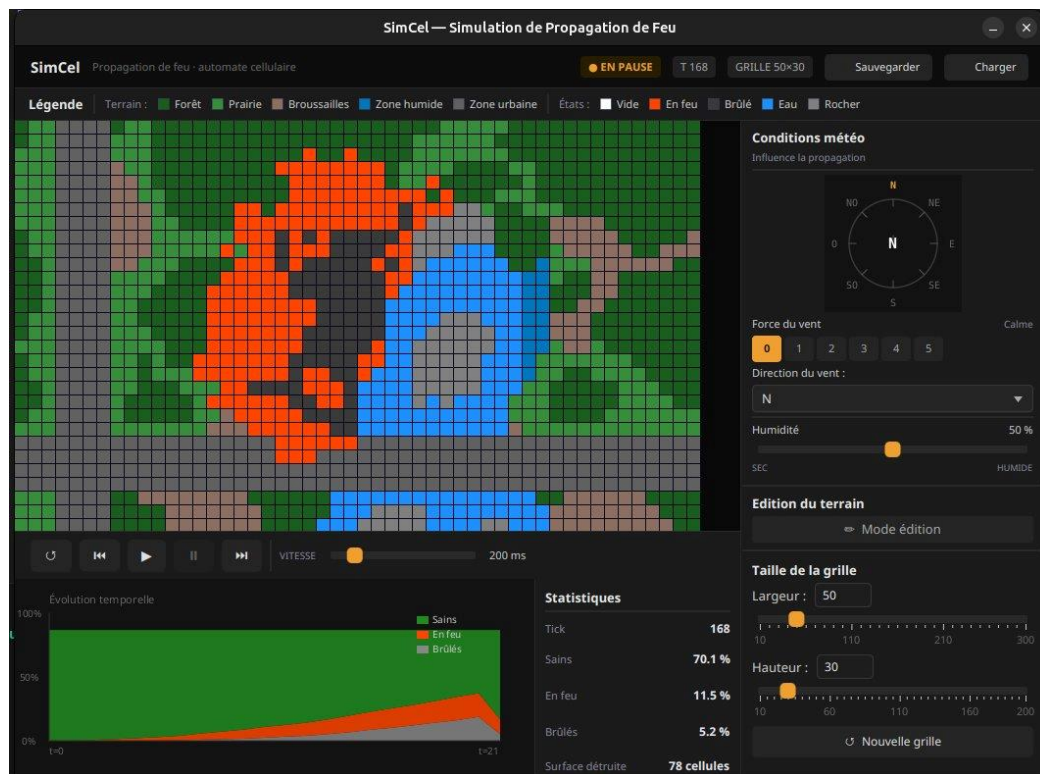


Fig. 2 — Propagation visible (T 168) : zones brûlées (gris), zones en feu (orange), zones saines (vert)

B. Diagramme de cas d'utilisation

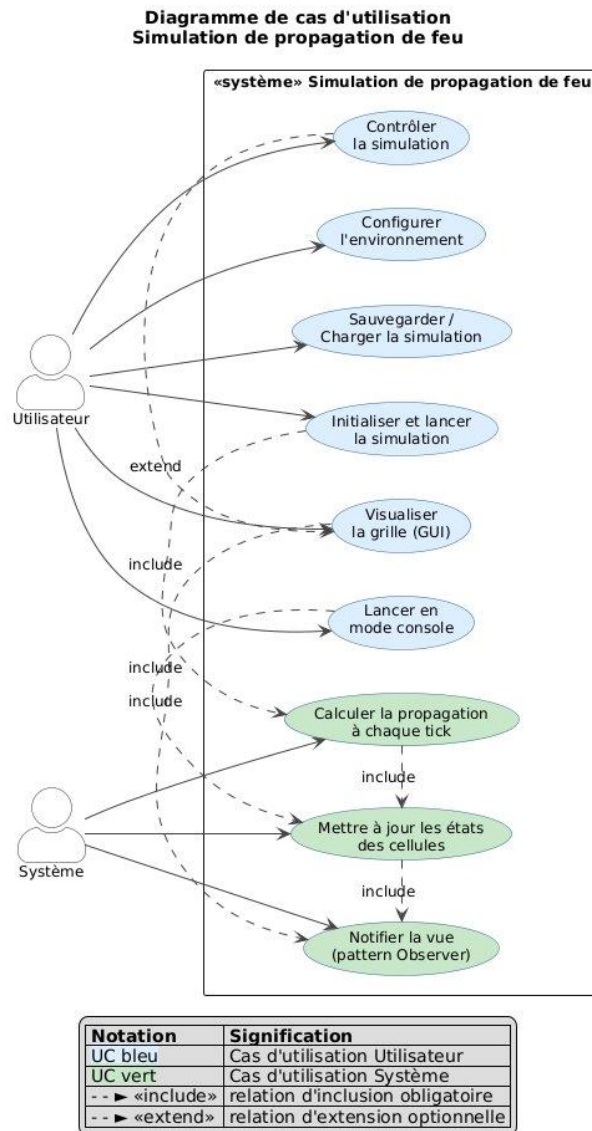


Fig. 3 — Diagramme de cas d'utilisation (M0)

C. Diagramme d'états-transitions

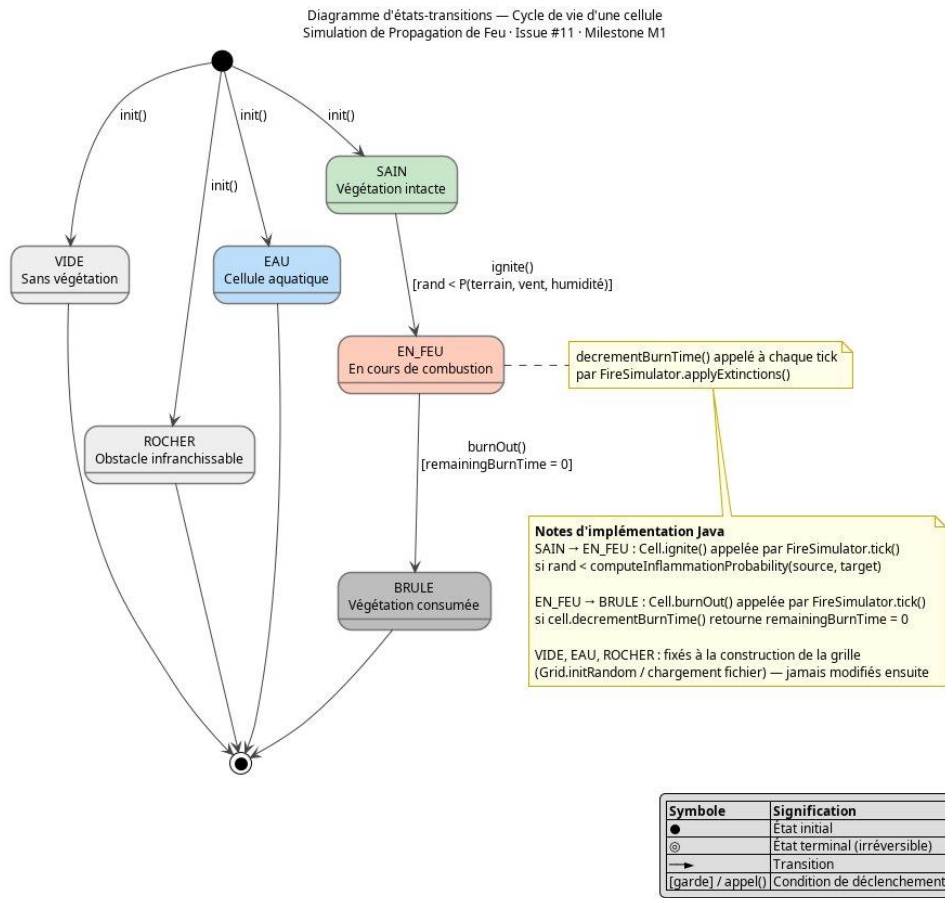


Fig. 4 — Cycle de vie d'une cellule (M1)

D. Diagramme de séquence

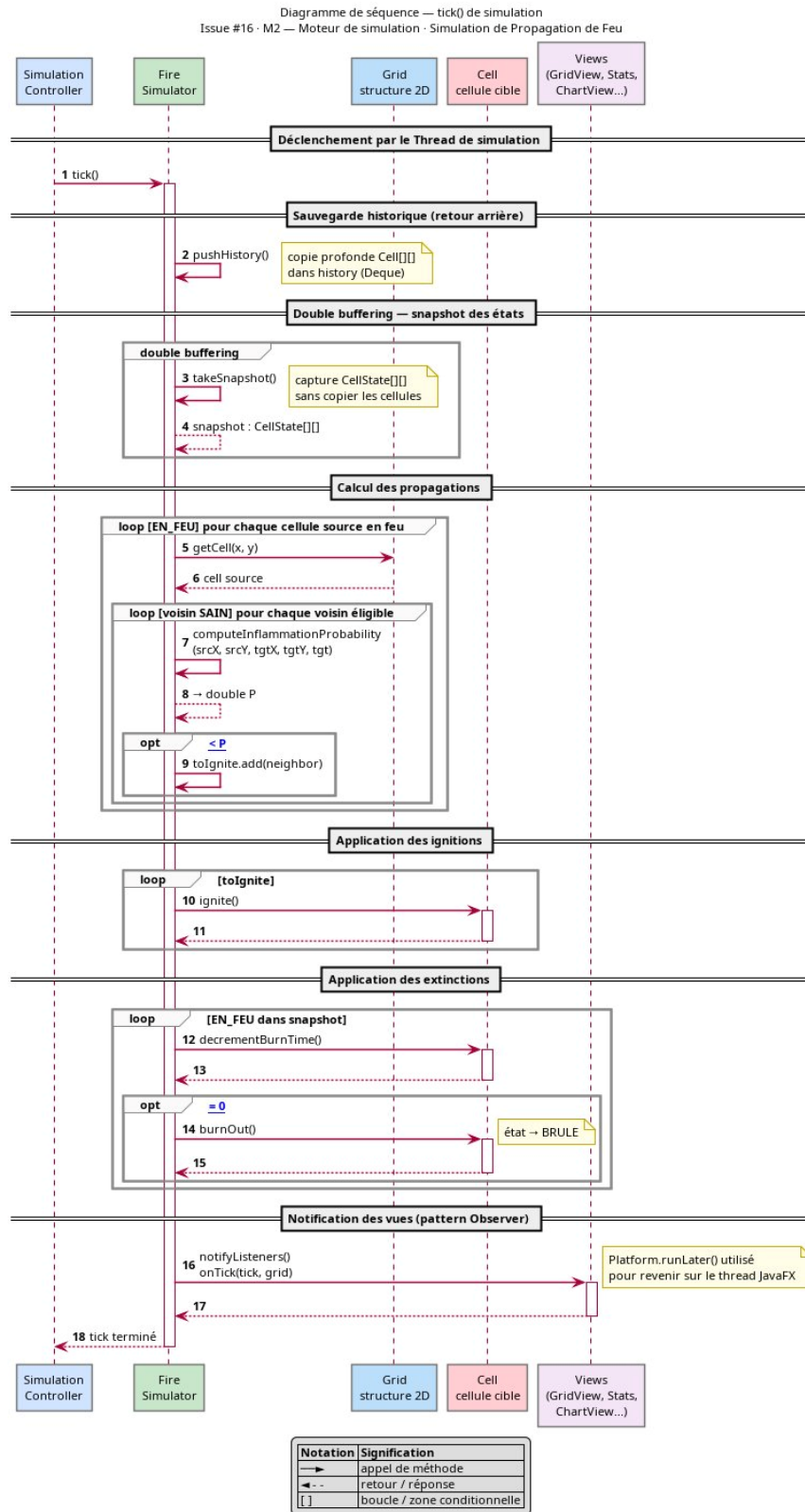


Fig. 5 — Déroulement d'un tick de simulation (M2)

E. Diagramme de classes

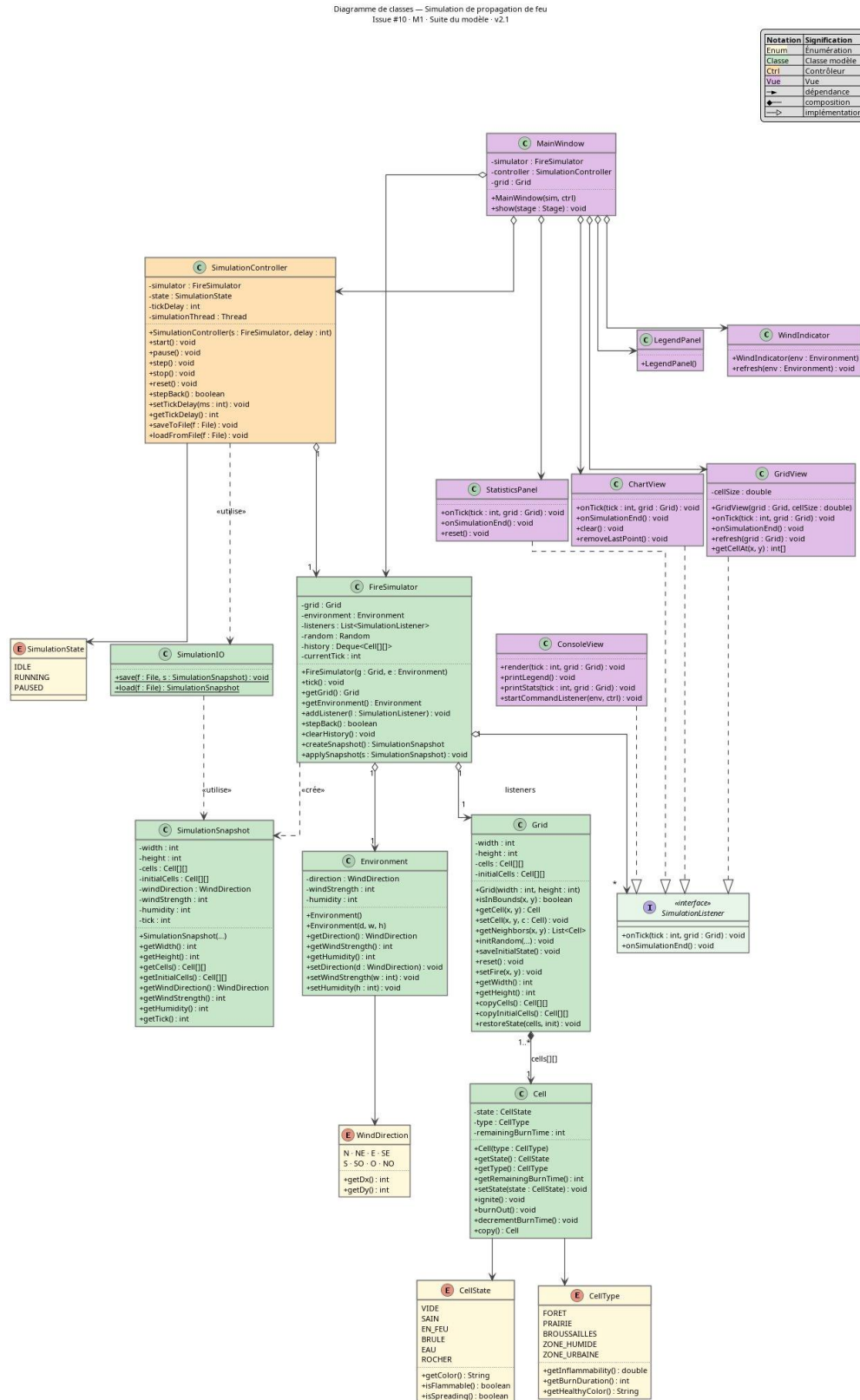


Fig. 6 — Diagramme de classes final (v2.1)